COMP 3800SEF /3820SEF /S380F /S380W Lecture 10: Dependency Injection (DI), Aspect-Oriented Programming (AOP)

Dr. Flora ZHANG

School of Science and Technology

Hong Kong Metropolitan University

Overview of this lecture

- *Decoupling* an interface from its implementation
- Dependency Injection (DI)
  - Construction injection
  - Setter injection
- Spring IoC (Inversion of Control) Container
- Spring Bean
  - Inner beans
  - Wiring Collections: List, Set, Map
  - Autowiring
- Aspect-Oriented Programming (AOP)

Jakarta Enterprise Bean (EJB)

- Class libraries only support the sharing of OO code.
- Jakarta EE has EJBs, which support the sharing of code and data.
  - Different OO applications can share states of objects running on the server.
- Jakarta EE supports many services through EJBs including transactions processing, Jakarta persistence API (JPA), concurrency control, naming and directory services (JNDI), security and web services.
- Programmer can invoke the services by calling the EJB's complex API.

Plain Old Java Object (POJO)

- The use of POJO allows Spring programmers to write code without too much changes to their coding style (EJBs do not allow their programmers to use them in a simple way).
- Spring and Hibernate make good use of POJOs.
- Under the influence from Hibernate and Spring, EJB 3 has improved over EJB 2 for being simpler.

POJO without an Interface

- The following class tries to perform a binary operation.
- The current binary operation is addition.
- If we want to change the binary operation to multiplication, we will need to change the bodies of two methods to change the operation.
- This code is not very maintainable.

```
public class Calculate {
   private long operate (long op1, long op2) {
      return op1 + op2;
   }

   private String getOpsName() {
      return " plus ";
   }
}
```

POJO with an Interface and Implementations

- To make the previous code more maintainable, we can define an **interface** for the binary operation.

```
public interface Operation {
   long operate (long op1, long op2);
   String getOpsName();
}
```

- We can have different **implementations** for the Operation interface, e.g., OpAdd and OpMultiply below.

```
public class OpAdd implements Operation {
   @Override
   public long operate(long op1, long op2) {
      return op1 + op2;
   }

   @Override
   public String getOpsName() {
      return " plus ";
   }
}
```

```
public class OpMultiply implements Operation {
   @Override
   public long operate(long op1, long op2) {
      return op1 * op2;
   }

   @Override
   public String getOpsName() {
      return " times ";
   }
}
```

Web application example

Web app example: lecture10

- URL: <base URL>/calEx1

```
@GetMapping("/calEx1")
public ModelAndView showCalForm1() {
    return new ModelAndView("cal", "calForm", new CalForm());
}

public static class CalForm {
    private long op1;
    private long op2;

    // getters and setters for op1, op2
}
```

cal.jsp

Calculation

Operand 1

0

Operand 2

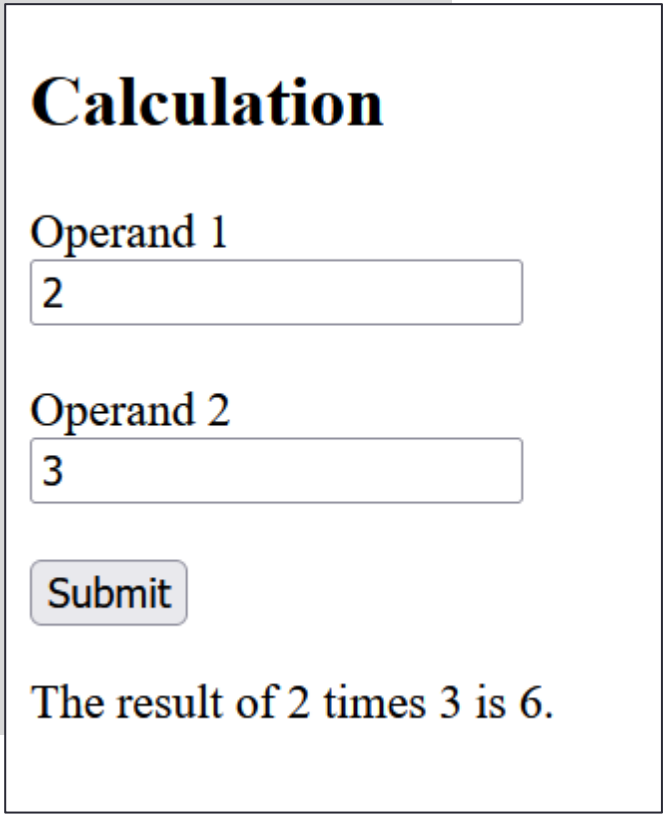
0

Submit

Caller choosing implementation

- Example URL: <base URL>/calEx1
- Just change the single statement in bold, we can switch to perform another operation, e.g., **Operation ops = new OpAdd();**

```
@PostMapping("/calEx1")
public ModelAndView cal1(CalForm calForm) {
    Operation ops = new OpMultiply();
    long op1 = calForm.getOp1();
    long op2 = calForm.getOp2();

    String result = "The result of " + op1
                + ops.getOpsName() + op2 + " is "
                + ops.operate(op1, op2) + ".";

    ModelAndView mav = new ModelAndView("cal");
    mav.addObject("calForm", calForm);
    mav.addObject("result", result);
    return mav;
}
```



Interface requires code change

- The use of interface can make a program more versatile and maintainable by choosing the desired implementation.
- Yet the caller must **change the code to switch implementations** in this interface solution.
  - Updating Java code requires re-compilation of the web application.
- Spring allows us to select implementation by **modifying the Java configuration class / XML configuration file** for the Spring application context.
  - Updating XML does not require re-compilation, but XML config can only be checked during runtime.
  - Java configuration provides type safety by compiler, and we only need to compile only the updated Java config class, instead of the web-app.

Adding Spring beans to XML configuration

- We can define Spring beans in an XML configuration file, as follows:

```
<bean id="multiply" class="hkmu.wadd.op.OpMultiply" />
<bean id="add" class="hkmu.wadd.op.OpAdd" />

<bean id="opsbean" class="hkmu.wadd.op.CalculateSpring">
   <property name="ops" ref="add" />
</bean>
```

```
public class CalculateSpring {
   private Operation ops;

   public void setOps(Operation ops) {
      this.ops = ops;
   }

   public long operate(long op1, long op2) {
      return ops.operate(op1, op2);
   }

   public String getOpsName() {
      return ops.getOpsName();
   }
}
```

- For each property, Spring will call the suitable setter method using the value of the attribute “ref” as an object reference.
  - setOps for property name ops
- We can switch between OpAdd and OpMultiply by setting this “ref” attribute to the bean “add” or the bean “multiply”

Adding Spring beans to Java config class

- Alternatively, we can define Spring beans in a Java config class:

hkmu.wadd.config

```
@Configuration
public class AppConfig {
    @Bean
    public Operation multiply() {
        return new OpMultiply();
    }

    @Bean
    public Operation add() {
        return new OpAdd();
    }

    @Bean
    public CalculateSpring opsbean() {
        CalculateSpring opsbean = new CalculateSpring();
        opsbean.setOps(add());
        return opsbean;
    }
}
```

```
public class CalculateSpring {
    private Operation ops;

    public void setOps(Operation ops) {
        this.ops = ops;
    }

    public long operate(long op1, long op2) {
        return ops.operate(op1, op2);
    }

    public String getOpsName() {
        return ops.getOpsName();
    }
}
```

Getting Spring bean in Controller method

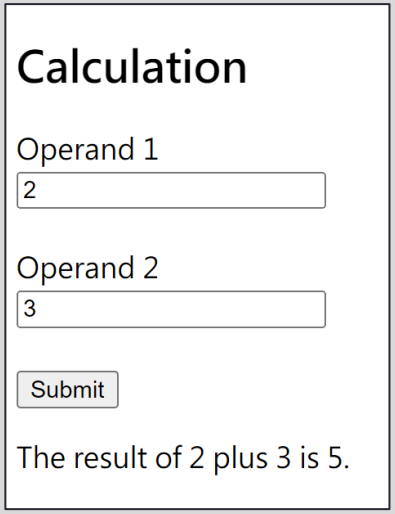
- Example URL: <base URL>/calEx2
- In a controller method, we can get the Spring bean "opsbean" using the **ApplicationContext** object and its **getBean()** function, as follows:

```
@Autowired
ApplicationContext context;
```

OR

```
ApplicationContext context;

@Autowired
public ExampleController(ApplicationContext context) {
    this.context = context;
    System.out.println("Application context "
                        +context.toString() +"loaded."); }
```

```
@PostMapping("/calEx2")
public ModelAndView cal2(CalForm calForm) {
    CalculateSpring cs = (CalculateSpring) context.getBean("opsbean");
    long op1 = calForm.getOp1();
    long op2 = calForm.getOp2();

    String result = "The result of " + op1
                + cs.getOpsName() + op2 + " is "
                + cs.operate(op1, op2) + ".";
    ModelAndView mav = new ModelAndView("cal");
    mav.addObject("calForm", calForm);
    mav.addObject("result", result);
    return mav;
}
```




```
2023-03-31T00:40:54.020+08:00  INFO 32064 --- [          main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1325 ms
Application context org.springframework.boot.web.servlet.context.AnnotationConfigServletWebServerApplicationContext@4c1f22f3, started on Fri Mar 31 00:40:52 HKT 2023 loaded.
```

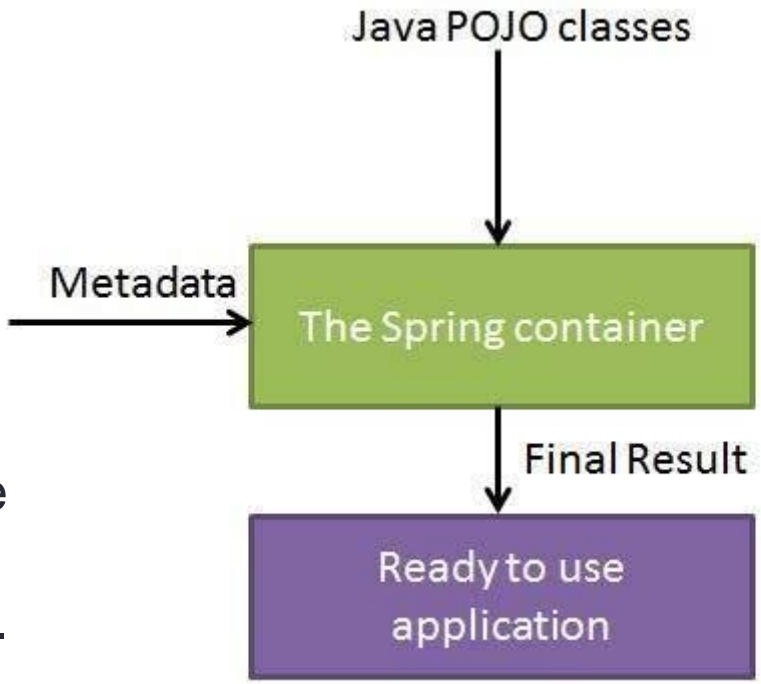
Dependency Injection (DI)

- The technique we've just shown is called Dependency Injection (DI).
- The dependency is no longer hard-coded in the caller but *injected* externally through a configuration file.
- DI is a form of **Inversion of Control (IoC).**
  - IoC is a design principle where it is the generic framework that calls into the custom code.
  - This inverts control as compared to traditional programming, where the custom code calls into reusable libraries to take care of generic task.
  - "*Don't call us; we'll call you*".

Spring IoC Container

- The **Spring IoC container** is at the core of the Spring Framework.
- The container will create the objects, wire them together, configure them, and manage their complete lifecycle from creation till destruction.

- The Spring IoC container uses **dependency injection (DI)** to manage the components that make up an application.
  - These objects are **Spring Beans**.
- The container gets its instructions on what Spring beans to instantiate, configure, and assemble by reading the provided configuration metadata (XML, Java annotations, or Java config class).



Spring Bean

- A Spring bean is instantiated, assembled, and otherwise managed by a Spring IoC container.
- These beans are created with the configuration metadata that you supply to the container, e.g., in a Java config class, we use `@Bean` annotation.
- The association made between Spring beans called **wiring**.

```
@Configuration
public class AppConfig {
    @Bean
    public Operation multiply() {
        return new OpMultiply();
    }

    @Bean
    public Operation add() {
        return new OpAdd();
    }

    @Bean
    public CalculateSpring opsbean() {
        CalculateSpring opsbean = new CalculateSpring();
        opsbean.setOps(add());
        return opsbean;
    }
}
```

More on Spring IoC Containers

- Spring application objects are created, wired, configured inside Spring containers.
- For standalone application, the following Spring application contexts are examples of Spring IoC Containers.

```
ApplicationContext context =
        new ClassPathXmlApplicationContext("beans.xml");

ApplicationContext context =
        new AnnotationConfigApplicationContext(BeanConfiguration.class);
```

- For web application, we can get the Spring application context using @Autowired:

```
@Autowired
ApplicationContext context;
```

Two types of Injection: Construction Injection

- Construction injection sets up a bean by its constructor.

```
public class CalPowerSpring {
    private int power;
    private Operation ops;

    public CalPowerSpring(int power, Operation ops) {
        this.power = power;
        this.ops = ops;
    }

    public String getOpsName() {
        return ops.getOpsName();
    }

    public long operate(long op1, long op2) {
        return ops.operate((long) Math.pow(op1, power), (long) Math.pow(op2, power));
    }

    public String getPower() {
        return String.valueOf(power);
    }
}
```

```
@Bean
public CalPowerSpring powerOpsBean() {
    return new CalPowerSpring(5, add());
}
```

Construction Injection (XML config)

- Construction injection sets up a bean by its constructor.

```
public class CalPowerSpring {
   …
   public CalPowerSpring(int power, Operation ops) {
       this.power = power;
       this.ops = ops;
    }
    …
}
```

```
<bean id="powerOpsBean"
        class="hkmu.wadd.op.CalPowerSpring">
   <constructor-arg value="5" />
   <constructor-arg ref="add" />
</bean>
```

- We use the **<constructor-arg>** tags to supply arguments to the constructor of CalPowerSpring (**in the same argument order**).
  - Use the **ref** attribute for passing a reference to an object.
  - Use the **value** attribute for passing a value directly.

Construction Injection (cont')

- Example URL: <base URL>/calEx3

```
@PostMapping("/calEx3")
public ModelAndView cal3(CalForm calForm) {
    CalPowerSpring cs = (CalPowerSpring) context.getBean("powerOpsBean");
    long op1 = calForm.getOp1();
    long op2 = calForm.getOp2();

    String result = "The result of " + op1 + "<sup>" + cs.getPower() + "</sup>"
                + cs.getOpsName() + op2 + "<sup>" + cs.getPower() + "</sup> is "
                + cs.operate(op1, op2) + ".";
    ModelAndView mav = new ModelAndView("cal");
    mav.addObject("calForm", calForm);
    mav.addObject("result", result);
    return mav;
}
```

Calculation

Operand 1

2

Operand 2

3

Submit

The result of 2^5 plus 3^5 is 275.

Two types of Injection: Setter Injection

- Setter injection sets up a bean by calling its setter methods.

```
@Bean
public CalculateSpring opsbean() {
    CalculateSpring opsbean = new CalculateSpring();
    opsbean.setOps(add());
    return opsbean;
}
```

OR

```
<bean id="opsbean"
        class="hkmu.wadd.op.CalculateSpring">
    <property name="ops" ref="add" />
</bean>
```

```
public class CalculateSpring {
    private Operation ops;

    public void setOps(Operation ops) {
        this.ops = ops;
    }

    public String getOpsName() {
        return ops.getOpsName();
    }

    public long operate(long op1, long op2) {
        return ops.operate(op1, op2);
    }
}
```

Example URL: <base URL>/calEx2

Inner Beans

- In Java, you have inner classes.
- In Spring, you also have inner beans. An inner bean is used only for a particular property of another bean.
- Below is an inner bean of OpAdd class injected by setter injection.

```
<bean id="opsWithInnerBean" class="hkmu.wadd.op.CalculateSpring">
   <property name="ops">
      <bean class="hkmu.wadd.op.OpAdd" />
   </property>
</bean>
```

- We can define a similar inner bean in Java Config, as follows:

```
@Bean
public CalculateSpring opsWithInnerBean() {
   CalculateSpring opsWithInnerBean = new CalculateSpring();
   opsWithInnerBean.setOps(new OpAdd());
   return opsWithInnerBean;
}
```

- Example URL: <base URL>/calEx4

Wiring Collections

- In previous examples, we switch between individual objects.
- In the following two examples, we switch between different collections of objects.

```
public class CollectionSpring {
    private Collection<Operation> ops;

    public String operateResult(long op1, long op2) {
        String result = "";
        for (Operation op : ops)
            result += "The result of " + op1
                      + op.getOpsName() + op2 + " is "
                      + op.operate(op1, op2) + ".</br>";
        return result;
    }

    public void setOps(Collection<Operation> ops) {
        this.ops = ops;
    }
}
```

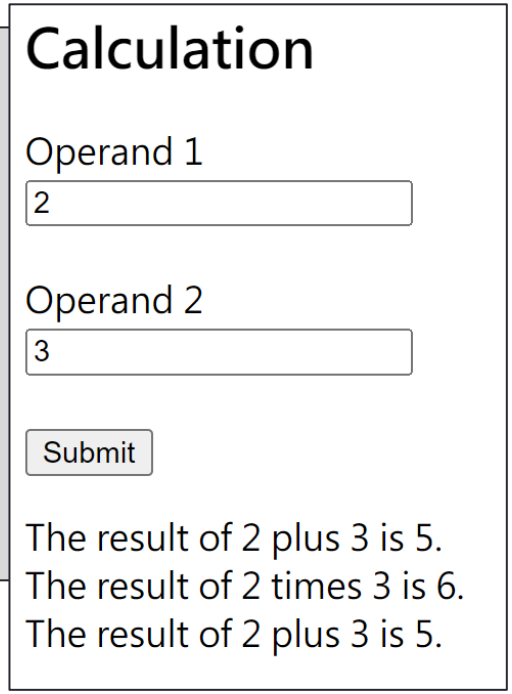
Wiring Collections: List

- Use **`Arrays.asList()`** to define a fixed-size list of `Operation` beans.

```
@Bean
public CollectionSpring listBean() {
    CollectionSpring listBean = new CollectionSpring();
    listBean.setOps(Arrays.asList(add(), multiply(), add()));
    return listBean;
}
```

```
@PostMapping("/list")
public ModelAndView calList(CalForm calForm) {
    CollectionSpring cs = (CollectionSpring) context.getBean("listBean");
    long op1 = calForm.getOp1();
    long op2 = calForm.getOp2();

    ModelAndView mav = new ModelAndView("cal");
    mav.addObject("calForm", calForm);
    mav.addObject("result", cs.operateResult(op1, op2));
    return mav;
}
```

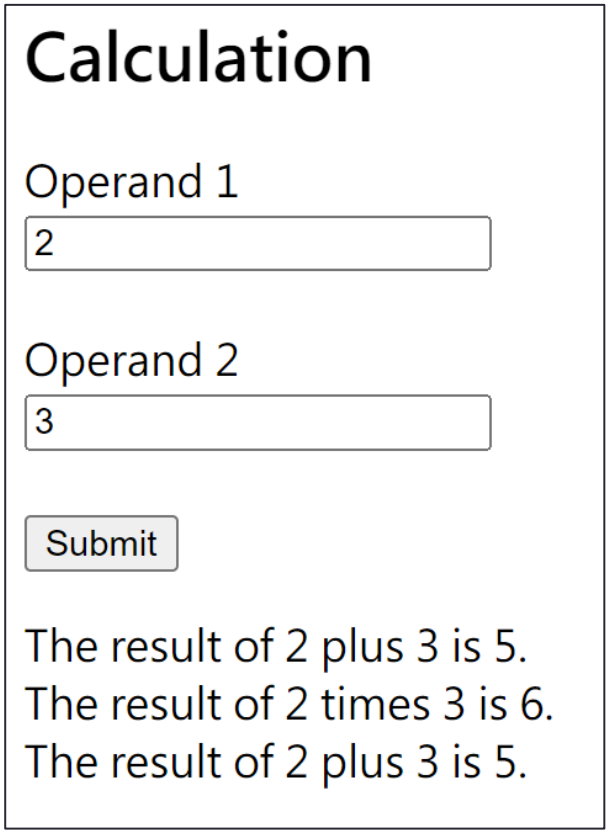
Example URL: <base URL>/list



Wiring Collections: List (XML config)

- A <list> holds possibly duplicated values.

```
<bean id="listBean"
        class="hkmu.wadd.op.CollectionSpring">
   <property name="ops">
      <list>
         <ref bean="add" />
         <ref bean="multiply" />
         <ref bean="add" />
      </list>
   </property>
</bean>
```

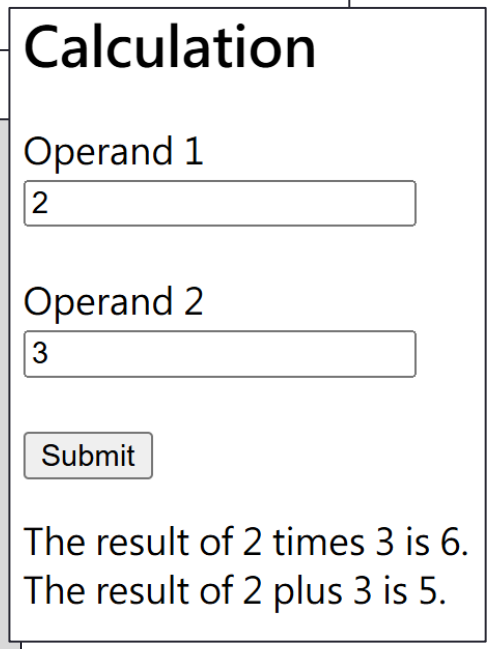


Wiring Collections: Set

- Use **HashSet** on a list to eliminate duplicated values automatically.

```
@Bean
public CollectionSpring setBean() {
    CollectionSpring setBean = new CollectionSpring();
    setBean.setOps(new HashSet<>(Arrays.asList(add(), multiply(), add())));
    return setBean;
}
```

```
@PostMapping("/set")
public ModelAndView calList(CalForm calForm) {
    CollectionSpring cs = (CollectionSpring) context.getBean("setBean");
    long op1 = calForm.getOp1();
    long op2 = calForm.getOp2();

    ModelAndView mav = new ModelAndView("cal");
    mav.addObject("calForm", calForm);
    mav.addObject("result", cs.operateResult(op1, op2));
    return mav;
}
```

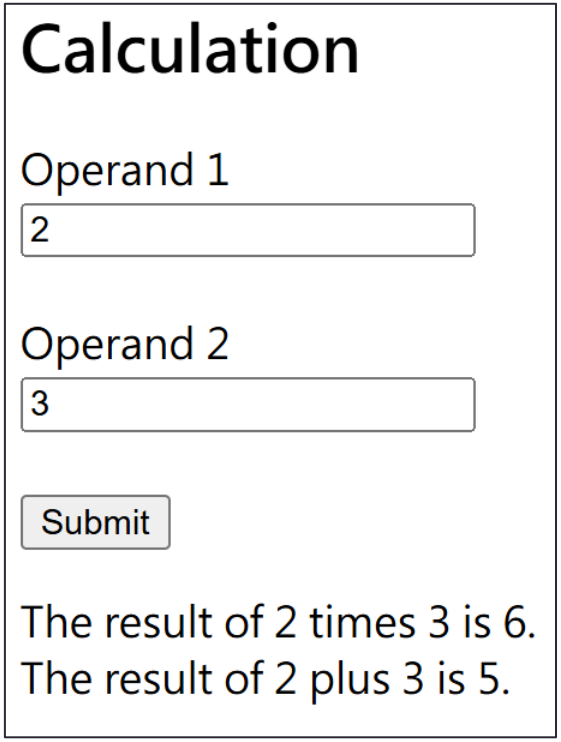


Example URL: <base URL>/set

Wiring Collections: Set (XML config)

- Use `<set>` to eliminate duplicated values automatically.

```
<bean id="setBean"
        class="hkmu.wadd.op.CollectionSpring">
   <property name="ops">
      <set>
         <ref bean="add" />
         <ref bean="multiply" />
         <ref bean="add" />
      </set>
   </property>
</bean>
```



Wiring Collections: Map

- If ops is a **Map<String, Operation>**:

```
@Bean
public CollectionSpring mapBean() {
    CollectionSpring mapBean = new CollectionSpring();
    Map<String, Operation> myMap = new HashMap<>();
    myMap.put("ADD", add());
    myMap.put("MULTIPLY", multiply());
    mapBean.setOps(myMap);
    return mapBean;
}
```

- **OR** for XML config, we can use **`<map>`**:

```
<bean id="mapBean" class="hkmu.wadd.op.CollectionSpring">
    <property name="ops">
        <map>
            <entry key="ADD" value-ref="add" />
            <entry key="MULTIPLY" value-ref="multiply" />
        </map>
    </property>
</bean>
```

Wiring Property-Value Pairs

- Property-value pairs are often called maps, dictionaries and associations.

```
public class JMSSource {
   private Properties sourceProps = null;
   public void setSourceProps(Properties sourceProps) {
      this.sourceProps = sourceProps;
   }
}
```

```
@Bean
public JMSSource jmsSource() {
   JMSSource jmsSource = new JMSSource();
   Properties p = new Properties();
   p.setProperty("numberOfThreads","10");
   p.setProperty("reuseThreads","true");
   p.setProperty("collectStatsEvery","hour");
   jmsSource.setSourceProps(p);
   return jmsSource;
}
```

OR

```
<bean name="jmsSource" class="some.package.JMSSource">
   <property name="sourceProps">
      <props>
         <prop key="numberOfThreads">10</prop>
         <prop key="reuseThreads">true</prop>
         <prop key="collectStatsEvery">hour</prop>
      <props>
   </property>
</bean>
```

Autowiring

```
@Bean
public Operation ops() {
    return new OpAdd();
}
```

- Given the bean on the right, the following two declarations are the same.

```
@Bean
public CalculateSpring opsbean() {
    CalculateSpring opsbean
            = new CalculateSpring();
    opsbean.setOps(add());
    return opsbean;
}
```

```
@Bean
@Autowired
public CalculateSpring opsbean(Operation ops) {
    CalculateSpring opsbean
                = new CalculateSpring();
    opsbean.setOps(ops);
    return opsbean;
}
```

- We can use **`@Autowired`** to annotate properties (instance variables), setters and constructors. The variable or the argument in the setter / constructor will be automatically wired (i.e., auto-wired).
- Here, opsbean's argument "ops" will be auto-wired to the bean "ops".
- Spring auto-wires with the most suitable bean; the autowiring mechanism is not purely matching by name or by type.
- This reference page shows how to disambiguate autowiring: https://docs.spring.io/spring-framework/reference/core/beans/annotation-config/autowired.html

Bean Scoping

- We can choose a scope for the bean to support a specific kind of sharing. The default is singleton.

```
@Bean
@Scope("prototype")
public Operation ops() {
    return new OpMultiply();
}
```

Scope	Meaning
singleton	One instance of the bean per Spring container
prototype	A new instance for every use of the bean
request	One instance per HTTP request
session	One instance per HTTP session
application	One instance for the application
websocket	One instance for a particular WebSocket session

Ref: https://docs.spring.io/spring-framework/reference/core/beans/factory-scopes.html

Aspect-Oriented Programming (AOP)

- The users use an application to perform the tasks they want, e.g., transfer money from one account to another. The users' tasks are the application's **main functions**.
- **Cross-cutting concerns** are not the user tasks they want to perform but are important nevertheless, e.g., logging, security and transaction management.
- **Aspect-Oriented Programming (AOP)** entails breaking programming logic into distinct parts, i.e., main functions and cross-cutting concerns.

More terminology:

- **Pointcuts** are the places in the main user tasks ***where*** cross-cutting concerns should be addressed.
- **Advices** are the code ***how*** to handle the cross-cutting concerns.
  - E.g., advices are the code to do logging, security, etc.
- An **aspect** consists of all the pointcuts and advices relating to a particular cross-cutting concern.

Without AOP vs. With AOP

Without AOP:

- The code for the application's main functions will be intermixed with the code for the cross-cutting concerns.
- The resulting code will be hard to maintain (because when you need to change the main functions, you will have to sieve through a large amount of unrelated cross-cutting concern code and vice versa).
- Also, it is harder to delegate the main function code and cross-cutting concern code to different programmers due to the undesirable intermixing.

With AOP:

- The codes of main function and cross-cutting concern are placed in separate files.
- It is the responsibility of the AOP framework to weave them together.

Example: Main function

- Consider the following money transfer operation, whose job is just to withdraw money from an account and deposit the same amount to another account.

```
void transfer(Account fromAcc, Account toAcc, int amount)
 throws Exception {
   if (! checkUserPermission(user)){
      throw new UnauthorizedUserException();
   }

   if (fromAcc.getBalance() < amount){
      throw new InsufficientFundsException();
   }

   fromAcc.withdraw(amount);
   toAcc.deposit(amount);
}
```

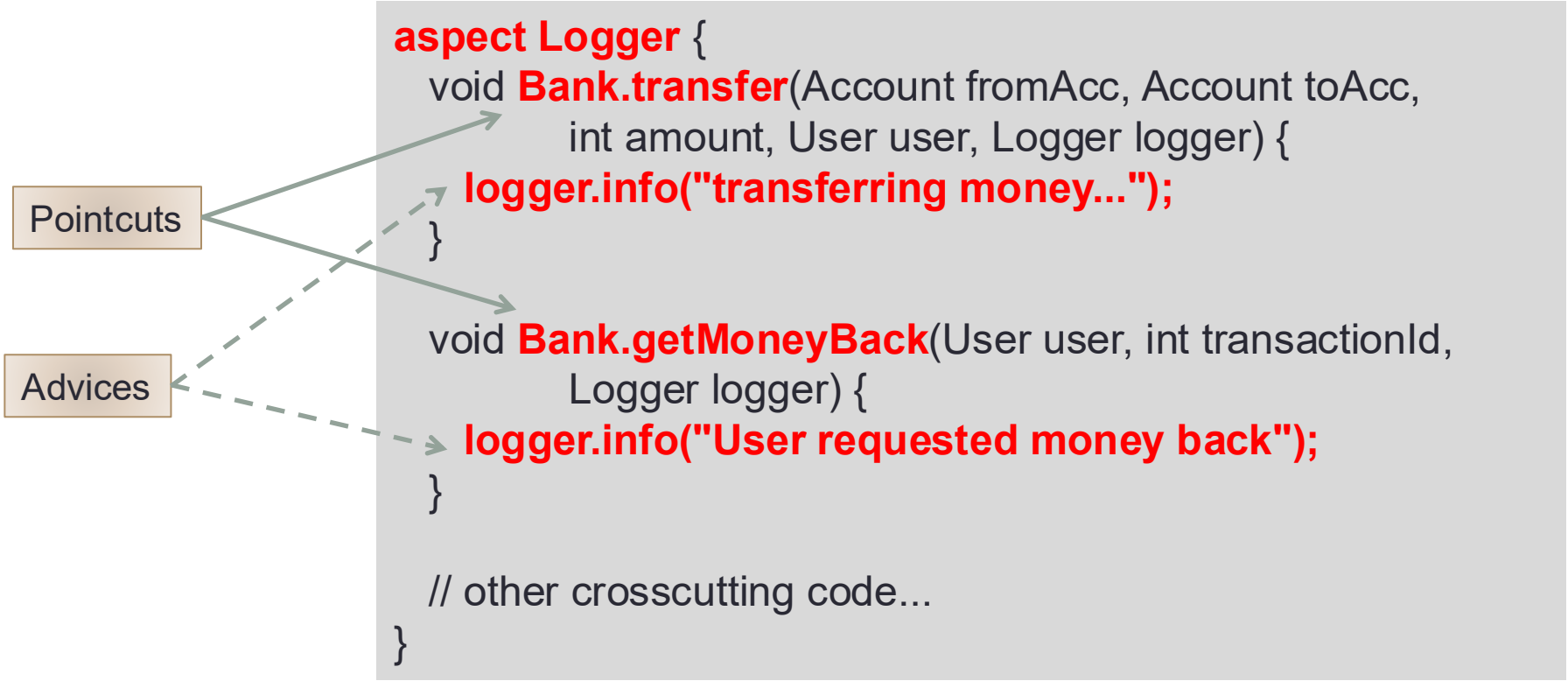
Example: Main function Intermixed with Logging

- After adding the required logging code, the original transfer code is cluttered by the logging code.

```
void transfer(Account fromAcc, Account toAcc, int amount)
 throws Exception {
   logger.info("transferring money...");
   if (! checkUserPermission(user)){
      logger.info("User has no permission.");
      throw new UnauthorizedUserException();
   }
   if (fromAcc.getBalance() < amount){
      logger.info("Insufficient Funds, sorry");
      throw new InsufficientFundsException();
   }
   fromAcc.withdraw(amount);
   toAcc.deposit(amount);
   logger.info("Successful transaction.");
}
```

Example: Extract Logging as an Aspect

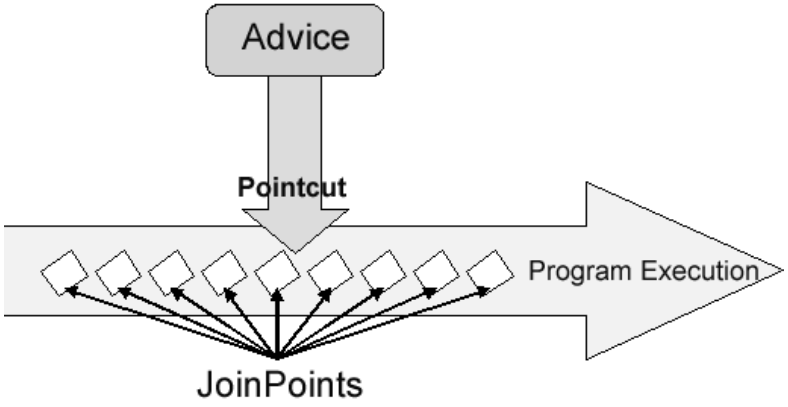
- With the following aspect (in pseudo-code), we don't need to litter the core code with **the cross-cutting concern logging** as on the last slide.



- A **pointcut** indicates where and when the cross-cutting concern applies.
- An **advice** describes what to do.

Steps of using AOP

- Write the **main functions** (user tasks).
- Write an **aspect** (e.g., security and logging) which has
  - **pointcut**: where and when the cross-cutting concern applies.
  - **advice**: the actions of the cross-cutting concern.
- The main functions, pointcuts and advices are separated clearly resulting in cleaner code.
- The framework, e.g., Spring, will **weave** the separated code together as an executable.
- Spring supports two approaches for custom aspects.
  - @AspectJ annotation
  - Schema-based



Defining Logging Aspect in Spring

- We use the @AspectJ annotation approach.

1. In build.gradle, add the dependencies for AspectJ:

```
implementation 'org.aspectj:aspectjrt'
implementation 'org.aspectj:aspectjweaver'
```

2. In the package `hkmu.wadd.aspect`, define the aspect class **`LoggingAspect`**:

```
@Aspect
public class LoggingAspect {
    // intercept any method execution in the class
    @Before("execution(* hkmu.wadd.op.Operation.*(..))")
    public void logMethodExecution(JoinPoint jp) {
        Logger logger = LoggerFactory.getLogger(LoggingAspect.class);
        logger.info("AOP logging: " + jp.toShortString());
    }
}
```

- Note that all other web app's classes are **unaffected** by AOP.

Defining Logging Aspect in Spring (cont')

3. Declare the Spring bean for **`LoggingAspect`**:

hkmu.wadd.config

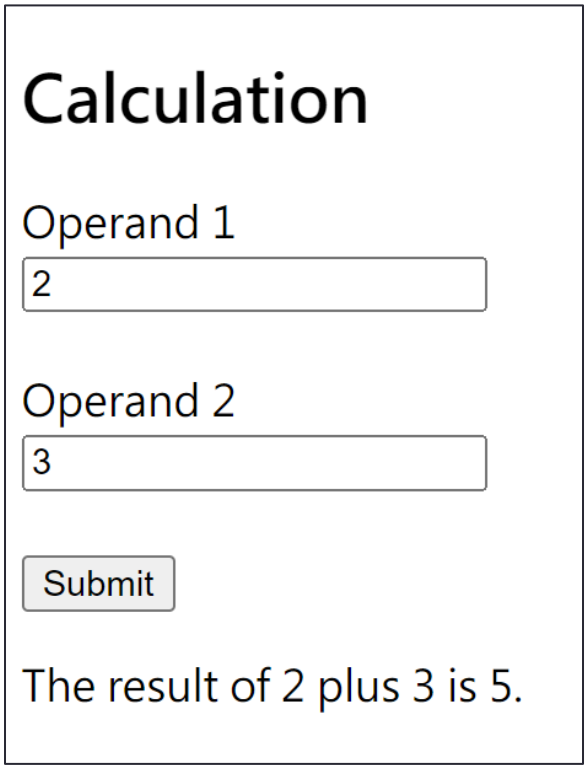
```
@Configuration
@EnableAspectJAutoProxy
public class AppConfig {

    …

    @Bean
    public LoggingAspect loggingAspect() {
        return new LoggingAspect();
    }
}
```

- @EnableAspectJAutoProxy enables support for handling components marked with AspectJ's @Aspect annotation.

Defining Logging Aspect in Spring (cont')

- Use the browser to access <base URL>/calEx2

- Perform the operation in the right.



- The following is displayed in Spring Boot's console:

```
ec-1] hkmu.wadd.aspect.LoggingAspect          : AOP logging: execution(OpAdd.getOpsName())
ec-1] hkmu.wadd.aspect.LoggingAspect          : AOP logging: execution(OpAdd.operate(..))
```